

PathEats

Year2 | Term3 | Team MHOB

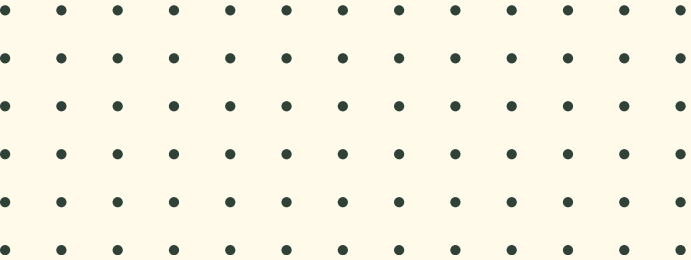
Course: Database Administration

Lecturer: MR.Ang Mengchhuong

Team members:

- Leng Layhok
- Kong Leak Smey
- Keo Seavpav

Present by: Seavpav



Contents

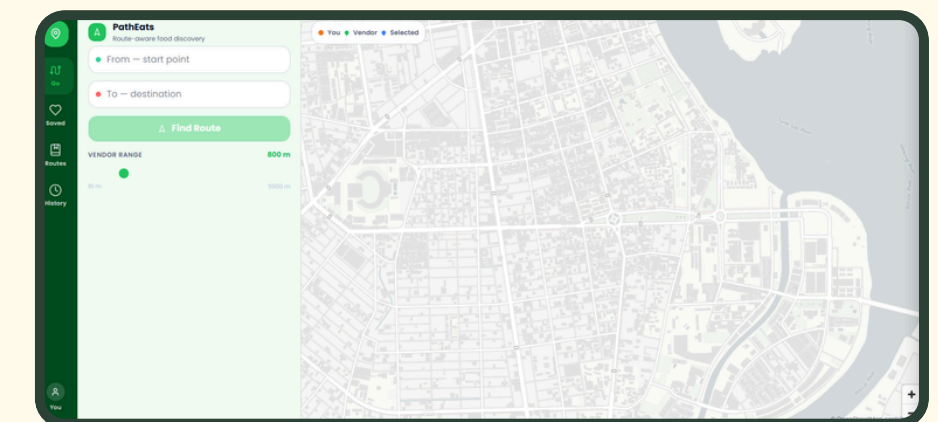
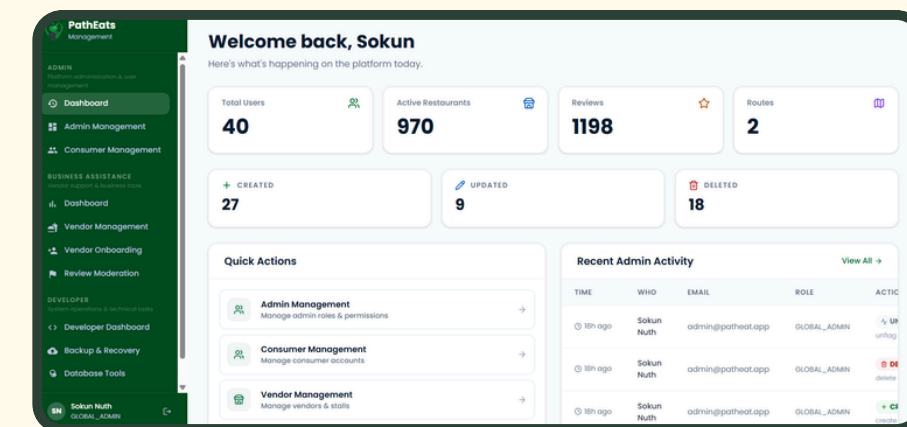
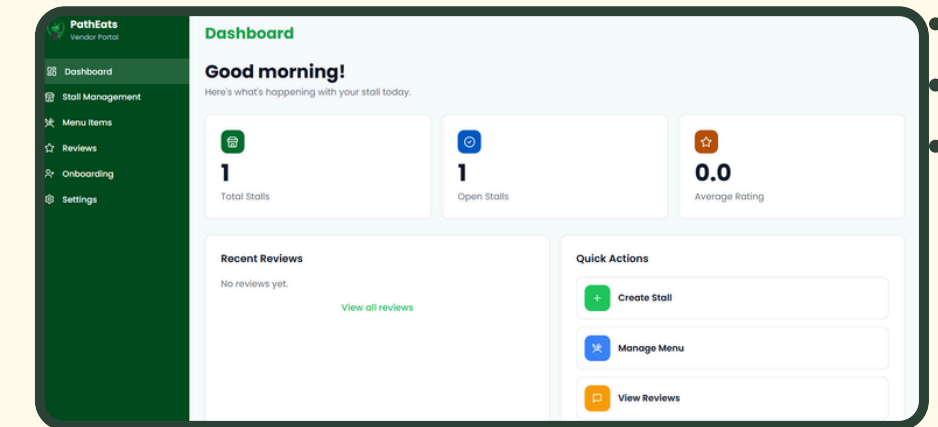
- > Introduction
- > Member Responsibility
- > ERD and Relation model
- > User Privileges & Access Control
- > Role and capability separation
- > Query optimization
- > Connection Pool
- > Indexing strategy
- > PostGIS filtering and pagination
- > Backup and Recovery

Introduction

PathEats helps consumers discover nearby food places while vendors manage their stalls and reusable menu catalogues.

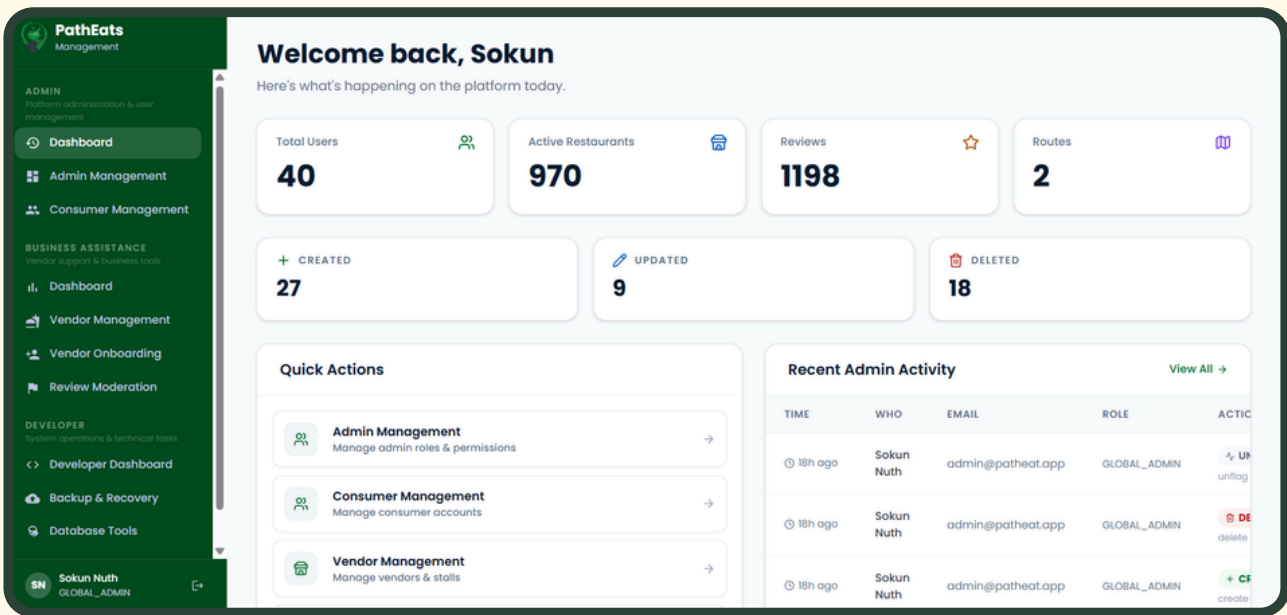
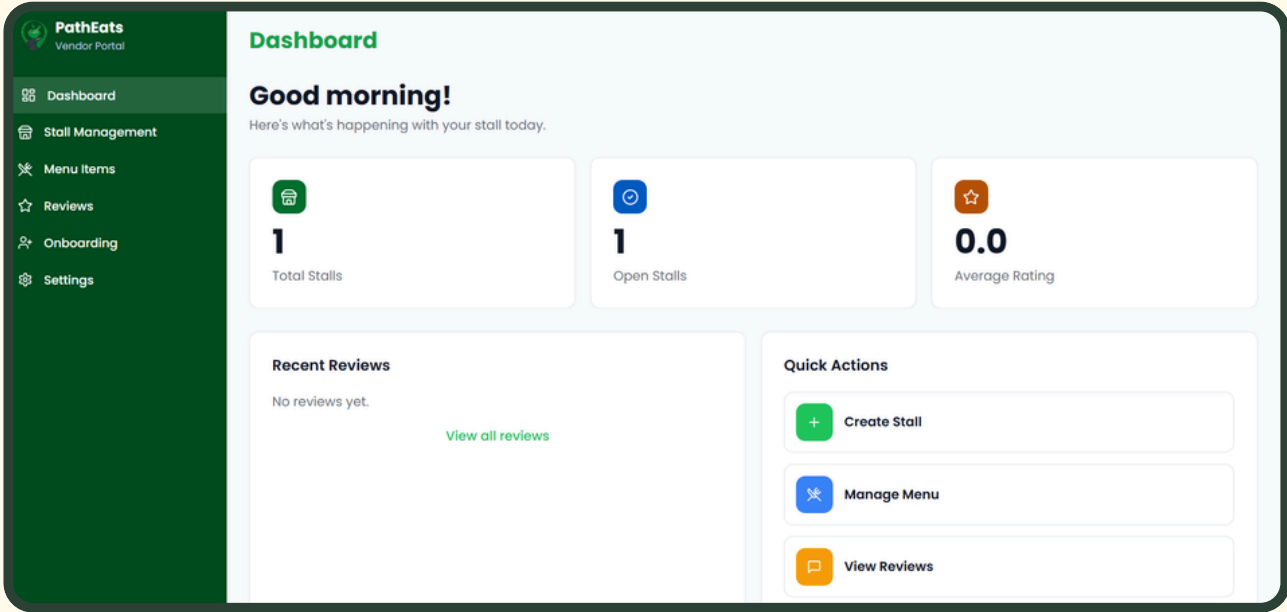
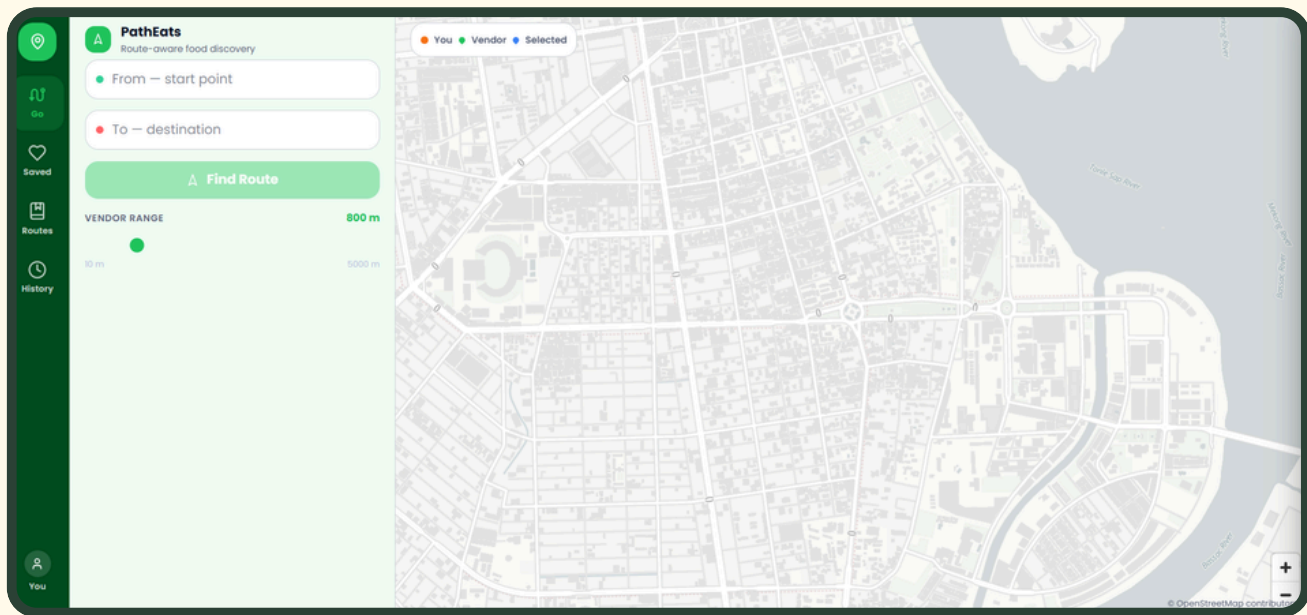
System Overview:

- Use PostGIS extension.
- Roles: Consumer, Vendor, Business Assistance, Global Admin and Developer Admin.
- Supabase Storage holds image objects separately.

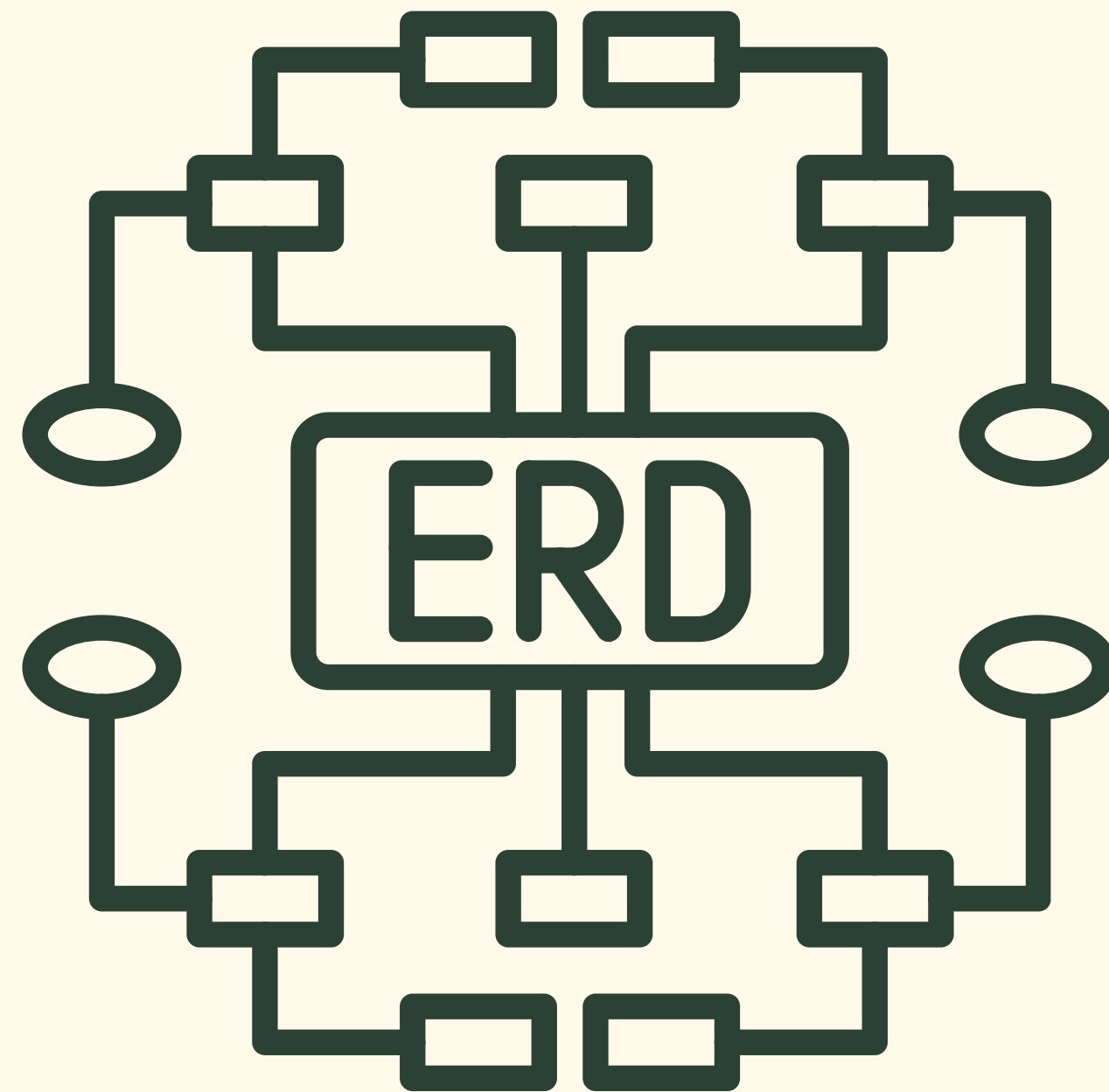


Member Responsibility

- Keo Seavpav: Vendor Side and Business Assistance Side
- Leng Layhok: Developer Side and Global Admin Side.
- Kong Leak Smey: Consumer Side



ERD and Relational model



User privileges and access control

```
const token = authHeader.split(" ")[1];
const decoded = jwt.verify(token, JWT_SECRET);
const { rows } = await db.query(
  `SELECT u.id, u.email, u.role_scope, u.role_id::text, u.is_banned,
    r.name AS role_name, r.base_scope, r.table_privileges,
    r.system_capabilities, r.grant_option, r.is_system
  FROM users u
  JOIN "role" r ON r.id = u.role_id
  WHERE u.id = $1
  LIMIT 1`,
  [decoded.sub]
);
```

name	text	table_privileges	jsonb
GLOBAL_ADMIN		{ "role": ["SELECT", "INSERT", "UPDATE", "DELETE"], "users": ["SEL	
BUSINESS_ASSISTANCE		{ "users": ["SELECT", "INSERT", "UPDATE"], "places": ["SELECT", "I	
HOHO		{ "users": ["SELECT", "INSERT", "UPDATE", "DELETE"], "user_prefe	
DEVELOPER_ADMIN		{ "system": ["MAINTENANCE", "BACKUP", "QUERY"], "all_tables": [	
VENDOR		{ "users": ["SELECT", "UPDATE"], "places": ["SELECT", "INSERT", "U	
CONSUMER		{ "users": ["SELECT", "UPDATE"], "places": ["SELECT"], "routes": [	



Role and capability separation

```
const decoded = jwt.verify(token, JWT_SECRET);
const { rows } = await db.query(
  `SELECT u.id, u.email, u.role_scope, u.role_id::text, u.is_banned,
    r.name AS role_name, r.base_scope, r.table_privileges,
    r.system_capabilities, r.grant_option, r.is_system
  FROM users u
  JOIN "role" r ON r.id = u.role_id
  WHERE u.id = $1
  LIMIT 1`,
  [decoded.sub]
);
```

Role access and system capabilities are checked separately.

```
// Developer operations
router.use(restrictToRoles("GLOBAL_ADMIN", "DEVELOPER_ADMIN"));
router.use("/backups", requireSystemCapability("BACKUP"));
router.use("/recovery", requireSystemCapability("RECOVERY"));
```



Query optimization

PathEats optimizes its common database paths using:

- 1. PostgreSQL connection pooling**
- 2. B-tree and PostGIS GiST indexes**
- 3. Early filtering and pagination**
- 4. Parameterized queries and targeted joins**

Connection Pool

```
const pool = new pg.Pool({
  connectionString: process.env.DATABASE_URL,
  ssl: process.env.NODE_ENV === 'production'
    ? { rejectUnauthorized: true }
    : { rejectUnauthorized: false },
  max: 20,
  idleTimeoutMillis: 30000,
  connectionTimeoutMillis: 10000,
  keepAlive: true,
});
```

**Requests reuse a shared pool
instead of opening a new
database connection each time.**

Indexing strategy

These indexes support spatial search, vendor ownership filters and available-menu retrieval.

```
-- Places, Menu items & categories
CREATE INDEX IF NOT EXISTS idx_places_location ON places USING GIST (location);
CREATE INDEX IF NOT EXISTS idx_places_owner ON places(owner_id);
CREATE INDEX IF NOT EXISTS idx_menu_items_owner ON menu_items(owner_id);
CREATE INDEX IF NOT EXISTS idx_place_menu_items_available
ON place_menu_items(place_id, is_available);
```

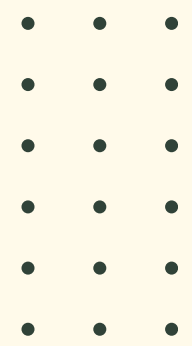
```
CREATE UNIQUE INDEX IF NOT EXISTS idx_reviews_unique_active
ON reviews (place_id, user_id)
WHERE deleted_at IS NULL;
```

This prevents one consumer from having multiple active reviews for the same place.



Indexing strategy

Index	Support
GiST places(location)	Nearby and route-based place searches
places(owner_id)	Vendor-owned place filtering
menu_items(owner_id)	Vendor catalogue filtering
place_menu_items(place_id, is_available)	Available menu lookup
Active-review unique index	Prevents duplicate active reviews



PostGIS filtering and pagination

```
const routeGeoJSON = JSON.stringify({
  type: "LineString",
  coordinates: routePoints.map(([lat, lng]) => [lng, lat]),
});
```

```
const conditions = [
  `p.status = 'active'`,
  `p.is_open = TRUE`,
  `p.deleted_at IS NULL`,
  `ST_Distance(p.location::geography, ST_GeomFromGeoJSON($1)::geography) <= $2`,
  `($3::text IS NULL OR pc.name = $3)`,
  `($4::int IS NULL OR p.price_range <= $4)`,
  `($5::text IS NULL OR p.name ILIKE $5 OR mi_search.names IS NOT NULL)`,
  `($6::boolean = FALSE OR ${OPEN_NOW_SQL})`,
];
```

PathEats converts coordinates to longitude–latitude order, filters inside PostgreSQL and returns only the requested page.

Backup Recovery

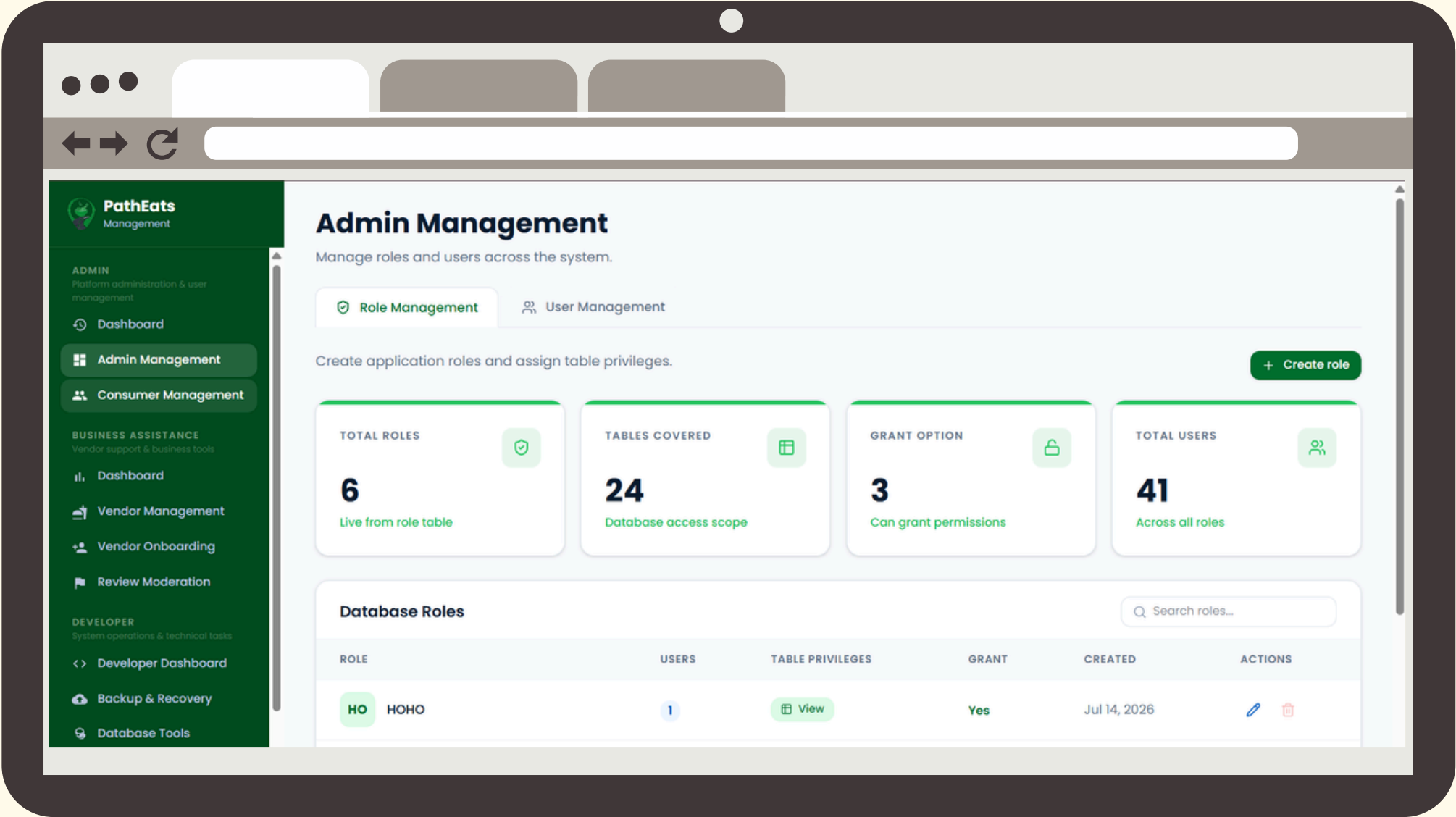
```
await runPostgresTool("pg_dump", [  
  "--format=custom",  
  "--no-owner",  
  "--no-privileges",  
  "--no-comments",  
  "--file",  
  dumpPath,  
  ...scopeArgs,  
]);
```

Backup

Recovery

```
await runPostgresTool("pg_restore", [  
  "--clean",  
  "--if-exists",  
  "--exit-on-error",  
  "--single-transaction",  
  "--no-owner",  
  "--no-privileges",  
  tableArg,  
  "--use-list",  
  restoreList.listPath,  
  "--dbname",  
  postgresToolConfig().databaseName,  
  dumpFilePath,  
]);
```


Demo



Thank you